

P0811R1: Well-behaved interpolation for numbers and pointers

Audience: LEWG, SG6; WG14

S. Davis Herring <herring@lanl.gov>

Los Alamos National Laboratory

February 9, 2018

Introduction

The simple problem of computing a value between two other values is surprisingly subtle in general. This paper proposes library functions to compute the midpoint between two integer, floating-point, or pointer values, as well as a more general routine that interpolates (or extrapolates) between two floating-point values.

These utilities are a pure library extension. With the exception of the pointer versions, they would be valuable and straightforward to implement in C (perhaps via type-generic macros); it would be appropriate to consult with WG14 about including them in the C standard library.

Midpoint

Integers

Computing the (integer) midpoint of two integer values via

$(a+b)/2$

can cause overflow for signed or unsigned integers as well as for floating-point values. [Java's binary search implementation](#) had this integer overflow bug for nearly a decade, and Mozilla had [the same issue](#) in its JavaScript implementation.

The standard alternative

$a+(b-a)/2$

works for unsigned integers (even if $b < a$). On a typical two's complement implementation where conversion from unsigned to signed preserves bit patterns, the library can provide the simple implementation

```
Integer midpoint(Integer a, Integer b) {  
    using U = make_unsigned_t<Integer>;  
    return Integer(U(a)+(U(b)-U(a))/2);  
}
```

that works for signed or unsigned Integer. Note that when $b == a+1$ or $b == a-1$ (without overflow), the result is a because of truncating division. This can be exploited to round half-integers up or down by supplying $a \geq b$ or $a \leq b$ respectively. (The simple $(a+b)/2$ always truncates half-integers towards 0, yielding $\min(a, b)$ when they differ by 1.)

Pointers

When a binary search over an array is implemented using pointers, the array size must not exceed PTRDIFF_MAX to avoid undefined behavior ([expr.add]/5). The library can also provide a function template

```
template<class T>
T* midpoint(T *a, T *b);
```

which is straightforward on common architectures but, it seems, **cannot be implemented** portably and efficiently in C++. As with integers, when the midpoint lies between two pointer values the one closer to *a* is chosen; for the usual case of *a*<*b*, this is compatible with the usual half-open ranges by selecting *a* when [*a*, *b*) is [*a*, *a*+1).

Floating-point types

Each of the midpoint formulas above can cause overflow for floating-point types; the latter can also produce results that are not correctly rounded (by rounding in the subtraction and the addition). A third choice

$a/2+b/2$

prevents overflow but is not correctly rounded for subnormal inputs (due to rounding each separately). The library can easily provide overloads of midpoint for floating-point types that is correctly rounded for all inputs by switching between these strategies:

```
Float midpoint(Float a, Float b)
{return isnormal(a) && isnormal(b) ? a/2+b/2 : (a+b)/2;}
```

Naming

The name *mean* is superior for the floating-point case, but for the other types (and the common application to binary search) the name *midpoint* used above is more appropriate. It would be possible to use both names (despite the possible confusion with the use of *mean* in [rand.dist]), but as it aims to replace the simple expression $a+(b-a)/2$ regardless of type, a single name seems best.

Linear interpolation

Both [obvious approaches](#) used in published implementations of floating-point linear interpolation have [issues](#):

1. $a+t*(b-a)$ does not in general reproduce *b* when $t==1$, and can overflow if *a* and *b* have the largest exponent and opposite signs.
2. $t*b+(1-t)*a$ is not monotonic in general (unless $a*b \leq 0$).

Lacking an obvious, efficient means of obtaining a correctly rounded overall result, the goal is instead to guarantee the basic properties of

1. *exactness*: $\text{linear}(a, b, 0) == a$ && $\text{linear}(a, b, 1) == b$
2. *monotonicity*: $(\text{linear}(a, b, t2) - \text{linear}(a, b, t1)) * (t2 - t1) * (b - a) \geq 0$
3. *boundedness*: $t < 0 \ || \ t > 1 \ || \ \text{isfinite}(\text{linear}(a, b, t))$
4. *consistency*: $\text{linear}(a, a, t) == a$

given that each argument *isfinite* (for monotonicity, *t1* and *t2* may also be infinite if $a \neq b$ and $t1 \neq t2$). These properties are useful in proofs of correctness of algorithms based on *linear*. When interpolating, consistency follows from the other properties, but it and monotonicity apply even when extrapolating.

To demonstrate the feasibility of satisfying these properties, a simple implementation that provides all of them is given here:

```
Float linear(Float a, Float b, Float t) {  
    // Exact, monotonic, bounded, and (for a=b=0) consistent:  
    if(a*b <= 0) return t*b + (1-t)*a;  
  
    if(t==1) return b; // exact  
    // Exact at t=0, monotonic except near t=1, bounded, and consistent:  
    const Float x = a + t*(b-a);  
    return t>1 == b>a ? max(b,x) : min(b,x); // monotonic near t=1  
}
```

Putting b first in the min/max prefers it to another equal value (i.e., $-0.$ vs. $+0.$), which avoids returning $-0.$ for $t==1$ but $+0.$ for other nearby values of t .

Whether it uses this implementation or not, the library can provide a function satisfying these mathematical properties.

Naming

The common (if abstruse) name `lerp` is avoided because it might suggest a restriction to t on $[0,1]$.

Wording

Midpoint

Add to the end of the synopsis in `[numeric.ops.overview]`:

```
// 29.8.14, least common multiple  
template<class M, class N>  
    constexpr common_type_t<M,N> lcm(M m, N n);  
  
A midpoint(A a, A b); // A arithmetic  
template<class T>  
    T* midpoint(T* a, T* b);  
  
}
```

Add a new subsection after `[numeric.ops.lcm]`:

29.8.15 Midpoint

`A midpoint(A a, A b);`

1. *Returns:* Half the sum of a and b . No overflow occurs. If A is an integer type and the sum is odd, the result is rounded towards a . If A is a floating-point type, at most one inexact operation occurs.
2. *Remarks:* An overload exists for each of `char` and all arithmetic types except `bool` being A .

```
template<class T>  
    T* midpoint(T* a, T* b);
```

1. *Requires:* a and b point to, respectively, elements $x[i]$ and $x[j]$ of the same array object x [*Footnote:* An object that is not an array element is considered to belong to a single-element array for this purpose; see

- [expr.unary.op]. A pointer past the last element of an array x of n elements is considered to be equivalent to a pointer to a hypothetical element $x[n]$ for this purpose; see [basic.compound]. — *end footnote*].
2. *Returns*: A pointer to $x[i+(j-i)/2]$, where the result of the division is truncated towards zero.

Linear interpolation

Add to the synopsis in [cmath.syn]:

```
long double fmal(long double x, long double y, long double z);
```

```
// 29.9.4, linear interpolation  
F linear(F a, F b, F t); // F floating-point
```

```
// 29.9.4, classification / comparison functions
```

Add a new subsection after [c.math.hypot3]:

29.9.4 Linear interpolation

```
F linear(F a, F b, F t);
```

1. *Returns*: $a+t(b-a)$. If each argument isfinite, the result satisfies these conditions:

1. $t < 0 \ || \ t > 1 \ || \ \text{isfinite}(\text{linear}(a, b, t))$
2. $\text{linear}(a, b, 0) == a \ \&\& \ \text{linear}(a, b, 1) == b$
3. $\text{linear}(a, a, t) == a$
4. $(\text{linear}(a, b, t_2) - \text{linear}(a, b, t_1)) * (t_2 - t_1) * (b - a) \geq 0$

The last condition applies additionally if $!\text{isnan}(t_1) \ \&\& \ !\text{isnan}(t_2) \ \&\& \ t_1 \neq t_2 \ \&\& \ a \neq b$.

2. *Remarks*: An overload exists for each floating-point type being F .